PARUL INSTITUTE OF ENGINEERING & TECHNOLOGY

DESIGN OF DATA STUCTURES

(303105201)

QUESTION BANK

PREPARD BY:- RUDRAM T KSHATRI

COMPUTER ENGG. DEPARTMENT

UNDER GUIDENCE OF:

PROF.K.HIMA BINDU

ASSOCIATE PROFESOR

COMPUTER ENGG. DEPARTMENT

UNIT 1 - UNIT 4

1. What is a data structure? Explain its importance. (2-3 Marks)

⇨ A **data structure** is a way of organizing, managing, and storing data so that it can be accessed and modified efficiently. It provides a systematic way to arrange data in memory, allowing for easy access and manipulation.

⇨ **Importance of Data Structures**:

1. **Efficiency**: Data structures enable efficient data management, reducing time complexity for operations such as searching, sorting, and updating data.
2. **Memory Management**: They help optimize the use of memory, allowing for efficient storage of large amounts of data.
3. **Data Organization**: They allow the organization of complex data in ways that make it easier to process and retrieve, which is essential in large-scale applications like databases, file systems, and network routing.

⇨ Data structures are fundamental for building reliable and scalable software systems.

2. Differentiate between linear and non-linear data structures. (4 Marks)

⇨

Aspect	Linear Data Structure	Non-Linear Data Structure
Definition	A data structure where elements are arranged in a sequential order, one after the other.	A data structure where elements are arranged in a hierarchical or interconnected manner.
Traversal	Traversal is done in a single level, from start to end (or vice versa).	Traversal can be done in multiple levels or directions.
Example	Arrays, Linked Lists, Stacks, Queues	Trees, Graphs
Complexity	Generally easier to implement and understand.	More complex due to hierarchical relationships.
Memory Usage	Uses contiguous memory (for arrays) or pointers (for linked lists).	Memory is often non-contiguous, with more dynamic relationships.
Use Case	Best suited for problems where data has a natural sequence.	Used when there is a need for hierarchical relationships or networks.

- **Linear Data Structure:** An array where elements are stored sequentially, like [1, 2, 3, 4, 5].
- **Non-Linear Data Structure:** A tree structure, where elements branch out, like in a family tree.

PREPARD BY:RUDRAM T KSHATRI

3. What are the primary operations that can be performed on a stack? (3-4 Marks)

⇨ A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle, meaning the last element added is the first one to be removed.

⇨ The primary operations performed on a stack are:

1. **Push**:
   - Adds (or inserts) an element to the top of the stack.
   - Example: If the stack is [1, 2, 3], performing a push(4) will result in [1, 2, 3, 4].
2. **Pop**:
   - Removes the top element from the stack.
   - Example: If the stack is [1, 2, 3, 4], performing a pop() will remove 4, resulting in [1, 2, 3].
3. **Peek**:
   - Returns the value of the top element without removing it.
   - Example: If the stack is [1, 2, 3, 4], peek() will return 4 without changing the stack.
4. **isEmpty**:
   - Checks if the stack is empty.
   - Returns true if the stack has no elements, otherwise false.
5. **isFull**:
   - Checks if the stack has reached its maximum capacity (if a fixed size is defined).

⇨ These operations are essential in many algorithms, such as evaluating expressions, implementing undo functionality, or traversing graphs.

4. Describe the difference between a stack and a queue. (4-5 Marks)

Feature	Stack	Queue
Data Structure Type	Linear	Linear
Access Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion (Push/Enqueue)	Elements are inserted (pushed) at the top of the stack.	Elements are inserted (enqueued) at the rear (tail) of the queue.
Removal (Pop/Dequeue)	Elements are removed (popped) from the top of the stack.	Elements are removed (dequeued) from the front (head) of the queue.
Order of Processing	The most recently added element is processed first.	The earliest added element is processed first.
Common Operations	Push (insert), Pop (remove), Peek (view top element)	Enqueue (insert), Dequeue (remove), Peek (view front element)
Use Cases	Undo/Redo functionality, expression evaluation, backtracking	Task scheduling, printer queue management, breadth-first search (BFS) in graphs

PREPARED BY:RUDRAM T KSHATRI

5. How does a circular queue differ from a regular queue? (3-4 Marks)

Feature	Regular Queue	Circular Queue
Structure	Linear, with front and rear pointers that do not wrap around.	Circular, where the rear connects to the front, forming a loop.
Space Utilization	Once the queue is full, no more elements can be added, even if there is empty space due to dequeued elements.	Efficiently reuses space by wrapping around when the rear reaches the last position, allowing insertion in previously used spots.
Front and Rear Movement	Front and rear pointers move in one direction and do not wrap around.	Both front and rear pointers wrap around to the start when they reach the end of the array.
Overflow Condition	Occurs when the rear reaches the last position, even if there is space at the front.	Overflow occurs only when all positions are occupied, making full use of available space.
Underflow Condition	Occurs when the queue is empty and no elements can be dequeued.	Similar underflow condition occurs when the queue is empty.
Common Use Cases	Simple task scheduling, buffering in systems where space isn't a concern.	Used in memory-constrained environments, buffering in network systems, and traffic management where efficient space utilization is needed.

6. Explain the concept of a linked list. How is it different from an array? (4-5Marks)

⇨ A **linked list** is a dynamic data structure made up of nodes, where each node contains two components: a **data** element and a **pointer** (or reference) to the next node in the sequence.

⇨ The linked list starts with a **head** node, and the last node in the list points to NULL, indicating the end of the list.

⇨ Linked lists can easily grow or shrink during runtime as elements are added or removed, making them ideal for applications where the number of elements is unpredictable.

⇨ There are different types of linked lists:

- A **singly linked list** has each node pointing to the next node.
- A **doubly linked list** has nodes that point both to the next and the previous node, allowing traversal in both directions.
- In a **circular linked list**, the last node points back to the first node, forming a loop.

⇨ In,an **array** is a static data structure where elements are stored in contiguous memory locations.

⇨ Unlike linked lists, arrays have a fixed size that is defined at the time of creation, and any addition or deletion requires shifting elements, making these operations slower.

⇨ Arrays, however, allow for **direct access** to elements via their index, making access times faster compared to linked lists, where elements must be accessed sequentially.

PREPARED BY:RUDRAM TKSHATRI

7. Difference between array and linked list. (3-4 Marks)

Feature	Linked List	Array
Definition	A linked list is a dynamic data structure where each element (node) contains a value and a pointer/reference to the next node.	An array is a static data structure where elements are stored in contiguous memory locations.
Size	Dynamic size; can grow or shrink during runtime as elements are added or removed.	Fixed size; the size is determined at the time of creation and cannot be changed.
Memory Allocation	Non-contiguous memory allocation; each node is scattered in memory, and nodes are linked via pointers.	Contiguous memory allocation; all elements are stored together in a single block of memory.
Access Time	Accessing an element requires traversing the list, leading to slower access time (O(n)).	Direct access to elements using indices, leading to faster access time (O(1)).
Insertion/Deletion	Easier and more efficient as insertion and deletion can occur at any position without shifting other elements (O(1) for head).	Insertion or deletion requires shifting elements, leading to slower operations (O(n)).
Use Cases	Useful when the size of the data set is unknown or changing frequently.	Ideal when fast access and fixed size are required, such as in scenarios with a predefined number of elements.

8. What are the main operations of a queue? Describe their time complexities. (3-4 Marks)

⇨ The main operations of a **queue** and their time complexities are as follows:

1. **Enqueue (Insertion)**:
   This operation adds an element to the **rear** (end) of the queue. The time complexity for this operation is **O(1)**, as it involves updating a pointer to the new element, without requiring any shifting of existing elements.
2. **Dequeue (Removal)**:
   This operation removes an element from the **front** of the queue. The time complexity for this operation is also **O(1)**, as it only involves updating the front pointer to the next element in the queue.
3. **Peek/Front**:
   This operation retrieves the element at the **front** of the queue without removing it. The time complexity is **O(1)**, as the front element can be accessed directly.
4. **isEmpty**:
   This operation checks whether the queue is empty or not. The time complexity is **O(1)**, since it typically involves comparing the front and rear pointers.

⇨ Overall, the core operations of a queue (enqueue, dequeue, peek, and isEmpty) have a constant time complexity of **O(1)**, making queues efficient for these operations.

PREPARD BY:RUDRAM T KSHATRI

9. What are the main operations of a stack? (2 Marks)

⇨ The main operations of a **stack** are:
⇨ **Push**: Adds an element to the top of the stack. The time complexity is **O(1)**.
⇨ **Pop**: Removes the element from the top of the stack. The time complexity is **O(1)**.
⇨ **Peek/Top**: Retrieves the element at the top of the stack without removing it. The time complexity is **O(1)**.
⇨ **isEmpty**: Checks whether the stack is empty. The time complexity is **O(1)**.

10. Describe the algorithm for evaluating postfix expressions using a stack. (3-4 Marks)

⇨ **To evaluate a postfix expression using a stack, follow these steps:**

⇨ Initialize an Empty Stack: Start with an empty stack to hold operands.

⇨ Read the Expression from Left to Right: Process each token (operand or operator) in the postfix expression sequentially.

⇨ Process Each Token:
- Operand: If the token is an operand (a number), push it onto the stack.
- Operator: If the token is an operator (e.g., +, -, *, /), perform the following steps:
- Pop: Pop the top two operands from the stack. These will be the operands for the operation.
- Compute: Apply the operator to these two operands. For example, if the operator is +, add the two operands.
- Push Result: Push the result of the operation back onto the stack.
- Finish: Once all tokens have been processed, the stack should contain only one element, which is the result of the postfix expression.

⇨ **<u>Example</u>**

For the postfix expression 5 3 + 2 *:

<u>Initialize an empty stack.</u>

Read 5: Push 5 onto the stack.
Read 3: Push 3 onto the stack.
Read +: Pop 3 and 5, compute 5 + 3 = 8, and push 8 onto the stack.
Read 2: Push 2 onto the stack.
Read *: Pop 2 and 8, compute 8 * 2 = 16, and push 16 onto the stack.

⇨ The stack now contains 16, which is the result of the postfix expression.
⇨ This algorithm ensures that each operator correctly applies to the most recent operands, following the postfix notation rules.

PREPARD BY:RUDRAM T KSHATRI

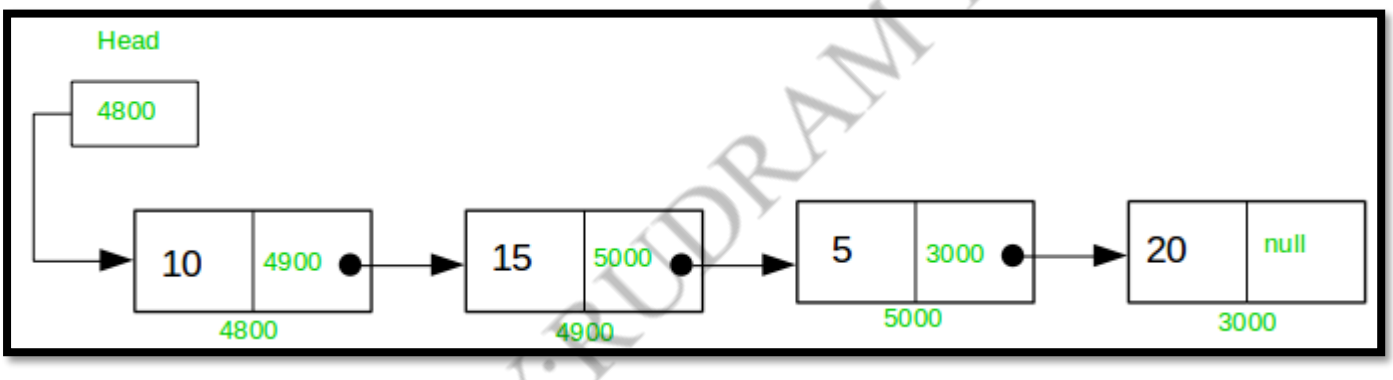
11. Explain the concept of a singly linked list and provide an example. (4 Marks)

⇨ A **singly linked list** is a type of linked list data structure where each node contains two components:

1. **Data**: The value or information stored in the node.
2. **Next**: A pointer or reference to the next node in the sequence.

⇨ The list starts with a **head** node, and each node points to the subsequent node, with the last node pointing to NULL, indicating the end of the list.
⇨ This allows traversal of the list in one direction, from the head to the last node.
⇨ **Example**
⇨ Consider a singly linked list with three nodes containing the values 10, 20, and 30:

- **Head Node**: Contains the value 10 and a pointer to the next node.
- **Second Node**: Contains the value 15 and a pointer to the third node.
- **Third Node**: Contains the value 5 and a pointer to the fourth node.
- **Fourth Node**: Contains the value 20 and a pointer to NULL, indicating the end of the list.



⇨ **Operations**
- **Traversal**: Start from the head and follow the next pointers until reaching NULL.
- **Insertion**: Add a new node by adjusting the pointers of the adjacent nodes.
- **Deletion**: Remove a node by updating the pointer of the previous node to bypass the node being deleted.

12. How does a doubly linked list differ from a singly linked list? (4-5 Marks)

Feature	Singly Linked List	Doubly Linked List
Node Structure	Each node has **data** and a **next** pointer to the next node.	Each node has **data**, a **next** pointer to the next node, and a **previous** pointer to the previous node.
Traversal	Can only be traversed in one direction (from head to end).	Can be traversed in both directions (from head to tail and tail to head).
Insertion and Deletion	Requires traversal to find the correct location; only the **next** pointers are updated. Insertion and deletion are less efficient.	Allows easier insertion and deletion as both **next** and **previous** pointers are updated. These operations are more efficient and straightforward.

PREPARED BY:RUDRAM T KSHATRI

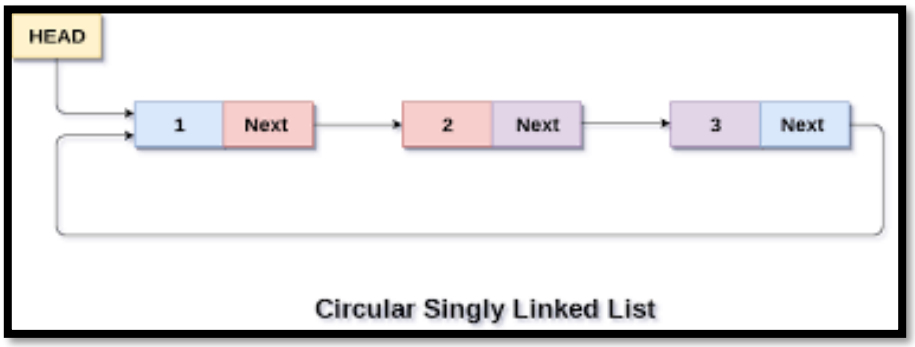
Memory Usage	Requires less memory per node (one pointer).	Requires more memory per node (two pointers).
Backward Traversal	Not possible due to the lack of a previous pointer.	Possible due to the **previous** pointer allowing traversal back to the head.
Complexity of Implementation	Simpler to implement with fewer pointers and less overhead.	More complex to implement due to managing two pointers (next and previous) for each node.
Efficiency of Operations	Insertion and deletion at known positions can be less efficient due to the need for traversal.	More efficient for operations at any position due to direct access to both the previous and next nodes.
Use Cases	Suitable for applications where forward traversal is sufficient and memory is a concern.	Useful in scenarios where bidirectional traversal is needed or where frequent insertions and deletions occur.

13. What is a circular linked list, and how is it implemented? (3 Marks)

- ⇨ A **circular linked list** is a variation of a linked list where the last node in the list points back to the first node, forming a circular structure.
- ⇨ This allows for continuous traversal of the list, as there is no end point; it loops back to the beginning.
- ⇨ **Implementation**

1. **Node Structure**:
   Each node in a circular linked list contains two components: **data** and a **next** pointer that references the next node in the sequence. In a singly circular linked list, the last node's next pointer points to the head node.
2. **Initialization**:
   - **Head Pointer**: Initialize a head pointer to manage the start of the list.
   - **Creating Nodes**: Create nodes with data and link them so that the last node's next pointer points back to the head.
3. **Example of Implementation**:
   - **Insertion**: To insert a node, adjust the pointers such that the new node links to the current head and update the last node's next pointer to the new node.
   - **Traversal**: Start from the head and keep following the next pointers until you return to the head, ensuring to handle the loop correctly to avoid infinite traversal.

- ⇨ **Example of Singly Circular Linked List:**



PREPARD BY:-RUDRAM T KSHATRI

14. What is linear search, and what is its time complexity? (2-3 Marks)

⇨ **Linear search** is a simple search algorithm used to find a specific element in a list.
⇨ The algorithm works by sequentially checking each element of the list until the desired element is found or the end of the list is reached.
⇨ **Steps for Linear Search:**

1. **Start at the beginning** of the list.
2. **Check each element** one by one.
3. **Compare** the current element with the target element.
4. **If a match is found**, return the index of the element.
5. **If the end of the list is reached** and no match is found, return an indicator that the element is not present.

⇨ **Time Complexity:**

- **Worst-Case Time Complexity**: **O(n)**, where n is the number of elements in the list. In the worst case, the algorithm must check every element in the list before concluding that the target is not present.
- **Best-Case Time Complexity**: **O(1)**, if the target element is at the very beginning of the list.

⇨ Linear search is straightforward and effective for small or unsorted lists but can be inefficient for large lists compared to more advanced search algorithms like binary search.

15. Explain binary search. How does it improve search efficiency compared to linear search? (4 Marks)
⇨ Binary search is an efficient algorithm for finding a target element in a sorted list.
⇨ It works by repeatedly dividing the search interval in half and comparing the target value to the middle element of the interval.

⇨ **Steps for Binary Search:**
- **Initialize two pointers:** low (starting at the beginning of the list) and high (ending at the last element of the list).
- Find the middle element of the current interval using the formula middle = (low + high) / 2.
- **Compare the middle element with the target value:**
- If the middle element is equal to the target, return its index.
- If the target is less than the middle element, adjust the high pointer to middle - 1 to search the left half.
- If the target is greater than the middle element, adjust the low pointer to middle + 1 to search the right half.
- Repeat steps 2 and 3 until the low pointer exceeds the high pointer or the target is found.

⇨ **Time Complexity:**
- **Worst-Case Time Complexity:** O(log n), where n is the number of elements in the list. Binary search halves the search space with each comparison, leading to a logarithmic reduction in the number of elements that need to be checked.
- **Best-Case Time Complexity:** O(1), if the target element is exactly at the middle of the list.

PREPARD BY:RUDRAM T KSHATRI

⇨ **Improvement over Linear Search:**

➢ **Efficiency**:
⇨ Binary search is significantly more efficient than linear search for large lists.
⇨ While linear search has a time complexity of O(n), binary search reduces the problem size exponentially with each step, resulting in O(log n) time complexity.
➢ **Suitability**:
⇨ Binary search requires the list to be sorted, which can be an overhead if the list is not already sorted.
⇨ However, for sorted lists, binary search is much faster compared to linear search, especially as the size of the list grows.

16. How does selection sort work? (2 Marks)
⇨ **Selection sort is a simple comparison-based sorting algorithm that works as follows:**

⇨ **Find the Minimum Element:** Traverse the unsorted portion of the list to find the smallest element.

⇨ **Swap**: Swap this minimum element with the first unsorted element in the list.

⇨ **Move the Boundary:** Consider the first element of the unsorted portion as sorted and repeat the process for the remaining unsorted portion.

⇨ **Steps**:
1. Start with the entire list.
2. Find the smallest element in the list.
3. Swap it with the first element of the list.
4. Move the boundary of the sorted portion one step to the right.
5. Repeat the above steps until the entire list is sorted.

⇨ **Example**:

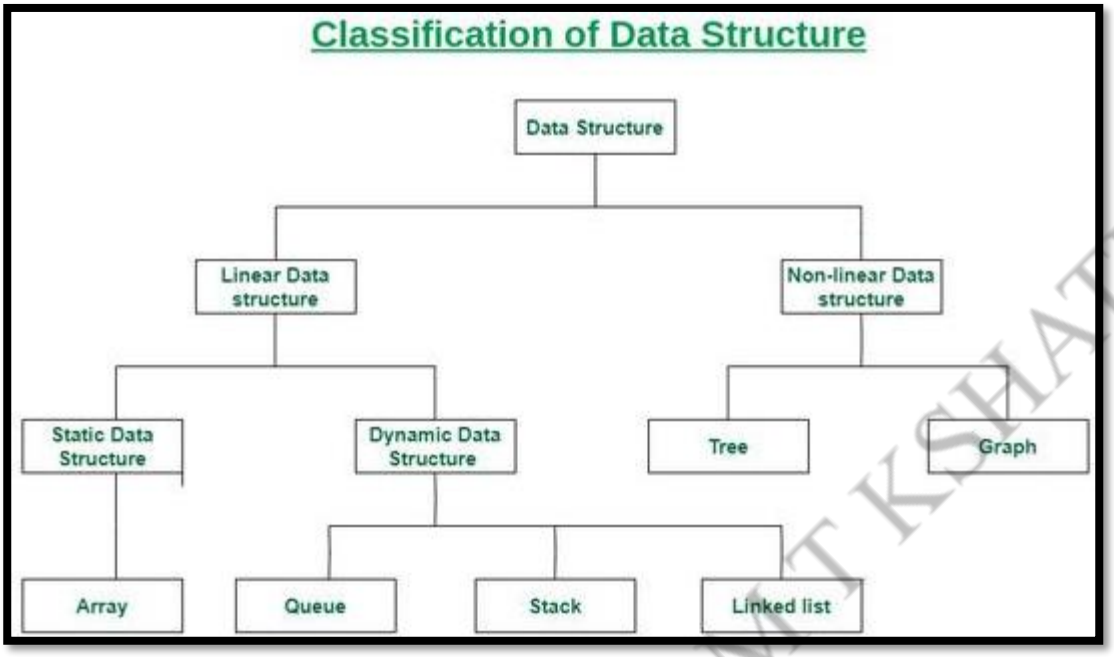
For a list [64, 25, 12, 22, 11]:

1. Find the smallest element (11) and swap it with the first element.
2. The list becomes [11, 25, 12, 22, 64].
3. Find the next smallest element (12) and swap it with the second element.
4. The list becomes [11, 12, 25, 22, 64].
5. Continue until the entire list is sorted.

⇨ Selection sort has a time complexity of O(n^2), making it inefficient for large lists compared to more advanced sorting algorithms.

PREPARD BY:RUDRAM T KSHATRI

17. Classify data structures with diagram. (4-5 Marks)



⇨ **Linear Data Structures**: These data structures store elements in a sequential manner.

- **Arrays**: A collection of elements identified by index.
- **Linked Lists**: A series of nodes where each node points to the next (and possibly previous) node.
  - **Singly Linked List**: Each node points to the next node.
  - **Doubly Linked List**: Each node points to both the next and previous nodes.
  - **Circular Linked List**: The last node points back to the head node.
- **Stacks**: A collection of elements with Last In, First Out (LIFO) access.
- **Queues**: A collection of elements with First In, First Out (FIFO) access.
  - **Circular Queue**: A queue where the last position is connected back to the first position.
  - **Priority Queue**: A queue where each element has a priority and is dequeued based on priority.

⇨ **Non-Linear Data Structures**: These data structures store elements in a hierarchical or interconnected manner.

⇨ **Trees**: Hierarchical data structures with a root node and child nodes.

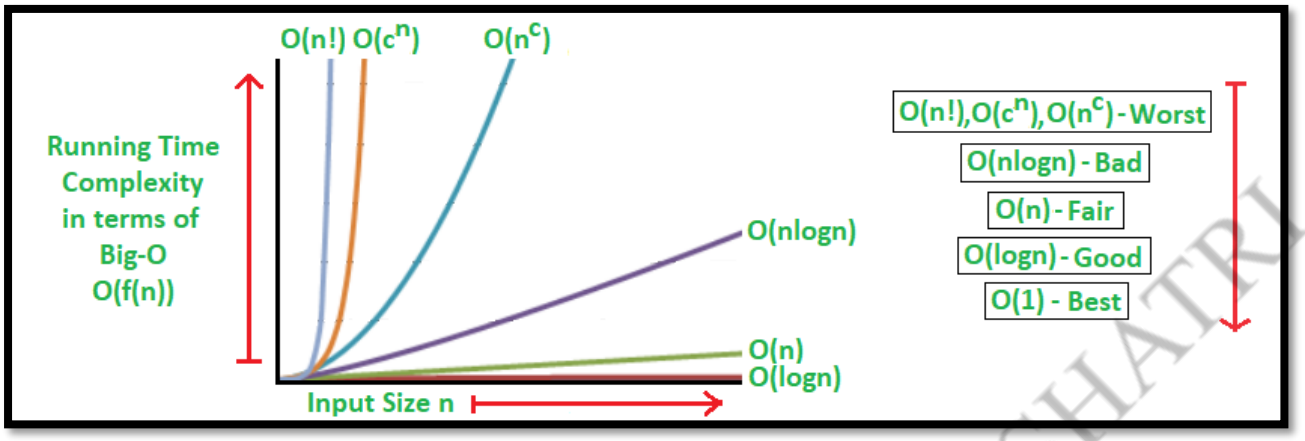
- **Binary Trees**: Each node has at most two children.
- **Binary Search Trees (BST)**: A binary tree where the left child is smaller and the right child is larger than the parent node.
- **AVL Trees**: A self-balancing binary search tree.
- **Red-Black Trees**: A self-balancing binary search tree with additional properties.
- **Heaps**: A complete binary tree where each node follows the heap property (min-heap or max-heap).

⇨ **Graphs**: A collection of nodes (vertices) and edges connecting them.

- **Directed Graphs**: Graphs where edges have a direction.
- **Undirected Graphs**: Graphs where edges do not have a direction.
- **Weighted Graphs**: Graphs where edges have weights.
- **Unweighted Graphs**: Graphs where edges do not have weights.

PREPARD BY:RUDRAM T KSHATRI

18. Interpret Big O complexity chart. (3 Marks)



⇨ **EXPLAINATION:-**

1. **O(1) - Constant Time**:
   - **Description**: The algorithm's runtime does not change with the size of the input.
   - **Example**: Accessing an element in an array by index.
2. **O(log n) - Logarithmic Time**:
   - **Description**: The runtime grows logarithmically as the input size increases.
   - This is often seen in algorithms that divide the problem in half with each step.
   - **Example**: Binary search in a sorted array.
3. **O(n) - Linear Time**:
   - **Description**: The runtime grows linearly with the input size.
   - Each element in the input contributes to the runtime.
   - **Example**: Iterating through an array to find an element.
4. **O(n log n) - Log-Linear Time**:
   - **Description**: The runtime grows in proportion to the input size multiplied by the logarithm of the input size.
   - Common in efficient sorting algorithms.
   - **Example**: Merge sort, Quick sort.
5. **O(n^2) - Quadratic Time**:
   - **Description**: The runtime grows quadratically with the input size.
   - This often occurs with algorithms that involve nested loops.
   - **Example**: Bubble sort, Selection sort.

19. Discuss Time complexity(2 Marks)

➢ **Definition**:
- The time complexity means how the problem take much time to solved and gest the answers.
- Time complexity measures how the runtime of an algorithm grows with the input size n.
- It is expressed using Big O notation (e.g., O(1), O(n), O(log n)).

➢ **Common Classes**:
- **O(1)**: Constant time – the runtime does not change with the input size.
- **O(log n)**: Logarithmic time – the runtime grows logarithmically as the input size increases.
- **O(n)**: Linear time – the runtime grows linearly with the input size.
- **O(n log n)**: Log-linear time – the runtime grows in proportion to n multiplied by log n.
- **O(n^2)**: Quadratic time – the runtime grows quadratically with the input size, often due to nested loops.

PREPARED BY:RUDRAM T KSHATRI

20. Describe sparse matrix. Find the address of A [2][1] if base address is 1024 for an integer array A[5][4] in row major order and word size is 2 byte. (3,4,5 Marks)

- ⇨ A sparse matrix is a matrix in which most of the elements are zero.
- ⇨ Storing all elements, including zeros, would be inefficient in terms of both memory and computation.
- ⇨ Instead, sparse matrices are often stored using specialized data structures to only keep track of non-zero elements and their positions, which significantly reduces storage requirements and computational overhead.

⇨ **Address Calculation for a Two-Dimensional Array in Row Major Order:-**
To find the address of an element A[2][1] in a two-dimensional array stored in row-major order, follow these steps:

⇨ **<u>Determine the dimensions and data:</u>**

Array Dimensions: A[5][4] (5 rows and 4 columns)
Base Address: 1024
Word Size: 2 bytes

⇨ **<u>Calculate the address using the row-major order formula:</u>**

Formula: Address of A[i][j] = Base Address + ((i * Number of Columns) + j) * Size of Each Element

Here, i = 2, j = 1, Number of Columns = 4, and Size of Each Element = 2 bytes.

⇨ **<u>Apply the formula:</u>**

Compute the offset: (i * Number of Columns) + j = (2 * 4) + 1 = 8 + 1 = 9
Compute the address: Base Address + (Offset * Size of Each Element) = 1024 + (9 * 2) = 1024 + 18 = 1042

⇨ **<u>Final Answer</u>**
The address of A[2][1] is 1042.

PREPARD BY:RUDRAM T KSHATRI

21. Given a two dimensional array A1(1:8, 7:14) stored in row-major order with base address 100 and size of each element is 4 bytes, find address of the element A1(4, 12). (4-5 Marks)

⇨ To find the address of the element A1(4, 12) in the given two-dimensional array A1 stored in row-major order, follow these steps:

⇨ **Given Information:**
Array dimensions: A1(1:8, 7:14)
Number of rows = 8 - 1 + 1 = 8
Number of columns = 14 - 7 + 1 = 8
Base address: 100
Size of each element: 4 bytes

⇨ **Row-Major Order Address Calculation**
Determine the Row and Column Indices:
The array is indexed from 1, so for A1(4, 12), i = 4 (row index) and j = 12 (column index).

⇨ **Calculate the Offset:**

The formula for the offset in row-major order is:
Offset=[(i−starting row index)×number of columns]+(j−starting column index)

⇨ **For the given array:**
Starting Row Index=1
Starting Column Index=7

Calculate the offset:

Offset=[(4−1)×8]+(12−7)
Offset=(3×8)+5
Offset=24+5=29

Calculate the Address:
Address=Base Address+(Offset×Size of Each Element)

Substituting the values:
Address=100+(29×4)
Address =100+116
Address=100+116
Address=216
Address=216
The address of the element A1(4, 12) is 216.

PREPARD BY:RUDRAM T KSHATRI

22. Define dynamic memory allocation? (2 Marks)

⇨ Dynamic memory allocation is a process in computer programming where memory is allocated at runtime, rather than at compile time.

⇨ This allows programs to request and release memory as needed, providing flexibility in managing memory usage.

❖ **Key Points:**

➢ **Runtime Allocation**: Memory is allocated when the program is running, not when it is being compiled.

➢ **Flexibility**: Enables the allocation of memory for data structures like arrays, linked lists, and objects based on the program's needs.

➢ **Functions**: In languages like C/C++, functions such as malloc(), calloc(), realloc(), and free() are used for dynamic memory management.

❖ **EXAMPLE**:-

```
int *arr = new int[10];        // Allocates memory for an array of 10 integers
```

23. Define referential structure? (2 Marks)

⇨ A **referential structure** is a type of data structure in which elements are connected or referenced through pointers or references rather than being stored contiguously in memory.

⇨ This allows for flexible and dynamic relationships between elements.

❖ **Key Points:**

- **Pointers or References**: Elements in a referential structure are typically connected using pointers (in languages like C/C++) or references (in languages like Java).
- **Dynamic Size**: Allows for dynamic resizing and easy insertion or deletion of elements since elements are not fixed in contiguous memory.

❖ **Common Examples**:

- **Linked Lists**: Nodes are linked using pointers; each node points to the next node.
- **Trees**: Nodes are connected such that each node can reference its child nodes, forming a hierarchical structure.
- **Graphs**: Nodes (vertices) are connected through edges, which can be represented using pointers or references.

24. Array is a heterogeneous data type. (True/False). Justify your answer. (2 Marks)

⇨ False, An array is not a heterogeneous data type; it is a homogeneous data type.

Justification:

⇨ Arrays are designed to store multiple elements of the same data type.

⇨ For example, an array of integers can only store integer values, and an array of floating-point numbers can only store float values.

Example:

```
int arr[5] = {1, 2, 3, 4, 5};
```

PREPARD BY:RUDRAM T KSHATRI

25. A m*n matrix which contains very few non-zero elements. A matrix contains more number of ZERO values than NON-ZERO values. Such matrix is known as ? (4-5 Marks)

⇨ A matrix that contains very few non-zero elements compared to zero values is known as a **sparse matrix**.

⇨ Key Characteristics of a Sparse Matrix:

1. **Many Zero Values**: Most of the elements in the matrix are zeros, and only a small fraction are non-zero.
2. **Efficient Storage**: Special data structures are often used to store sparse matrices efficiently, avoiding the waste of memory that would result from storing all zeros explicitly.
3. **Usage**: Sparse matrices are commonly used in fields like scientific computing, optimization, and graph algorithms where matrices with a high proportion of zero elements are common.

⇨ **Storage Techniques:**

To handle sparse matrices efficiently, several techniques are used, such as:

- **Compressed Sparse Row (CSR)**: Stores only non-zero elements and their row and column indices.
- **Compressed Sparse Column (CSC)**: Similar to CSR but stores column indices.
- **Coordinate List (COO)**: Stores a list of tuples with row index, column index, and value of non-zero elements.

<u>EXAMPLE</u>:-

$$\begin{matrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 2 & 0 \\ 0 & 3 & 0 & 0 \\ 4 & 0 & 0 & 0 \end{matrix}$$

⇨ This matrix has only 4 non-zero elements out of 16 total elements, making it a sparse matrix.

PREPARD BY:RUDRAM T KSHATRI

26. Convert infix to Postfix and Prefix(4-5 Marks)

a. (A + B) / C – D * E
b. P ^ Q ^ R + S / T
c. A * B – (C / D + (E – F)) ^ G

Solution:-

a. $(A + B)/C - D * E$

Expression Type	Conversion Steps	Result
Infix	$(A + B)/C - D * E$	
Postfix	1. Convert $(A + B)$ to `A B +` 2. Append `/ C` to get `A B + C /` 3. Convert `D * E` to `D E *` 4. Combine: `A B + C / D E * -`	`A B + C / D E * -`
Prefix	1. Reverse infix: `E * D - C / (B + A)` 2. Convert reversed to postfix: `E D * B A + C / -` 3. Reverse postfix: `- / + A B C * D E`	`- / + A B C * D E`

b. P ^ Q ^ R + S / T

Expression Type	Conversion Steps	Result
Infix	P ^ Q ^ R + S / T	
Postfix	1. Convert P ^ Q ^ R to `P Q R ^ ^` 2. Convert S/T to `S T /` 3. Combine: `P Q R ^ ^ S T / +`	`P Q R ^ ^ S T / +`
Prefix	1. Reverse infix: `T / S + R ^ Q ^ P` 2. Convert reversed to postfix: `T S / R Q ^ ^ P +` 3. Reverse postfix: `+ ^ ^ P Q R / S T`	`+ ^ ^ P Q R / S T`

c. $A * B - (C/D + (E - F))^G$

Expression Type	Conversion Steps	Result
Infix	$A * B - (C/D + (E - F))^G$	
Postfix	1. Convert $E - F$ to `E F -` 2. Convert C/D to `C D /` 3. Convert $CD/ + (EF-)$ to `C D / E F - +` 4. Convert $(CD/EF - +)^G$ to `C D / E F - + G ^` 5. Combine: `A B * C D / E F - + G ^ -`	`A B * C D / E F - + G ^ -`
Prefix	1. Reverse infix: `G ^ + / D C - F E * B A` 2. Convert reversed to postfix: `G E F - D C / + ^ B A *` 3. Reverse postfix: `- * A B ^ + / C D ^ G E F`	`- * A B ^ + / C D ^ G E F`

PREPARD BY:RUDRAM T KSHATRI

27. List applications of stack and Convert 2 * 3 / (2-1) + 5 * 3 infix expression into postfix format. Showing stacks status after every step in tabular form and evaluates that postfix notation. (4-5 Marks)

⇨ **Applications of Stacks**

1. **Function Call Management**: Used to manage function calls and returns in programming languages through call stacks.
2. **Expression Evaluation**: Helps evaluate expressions in postfix (Reverse Polish Notation) and prefix notations.
3. **Undo Mechanism**: Maintains a history of actions for undo operations in software applications.
4. **Syntax Parsing**: Used in compilers to parse syntax and manage parsing states.
5. **Backtracking Algorithms**: Assists in backtracking algorithms such as those used in puzzles and search problems.

Conversion to Postfix Notation

Step	Action	Stack	Postfix Expression	Explanation
1	Read 2		2	Operand added to postfix expression.
2	Read *	*	2	Operator * pushed onto stack.
3	Read 3	*	2 3	Operand added to postfix expression.
4	Read /	/ *	2 3 *	Operator / has higher precedence than *, so * is popped to postfix and / is pushed.
5	Read (	/ (	2 3 *	(is pushed onto the stack.
6	Read 2	/ (	2 3 * 2	Operand added to postfix expression.
7	Read -	/ (-	2 3 * 2	Operator - pushed onto stack inside (.
8	Read 1	/ (-	2 3 * 2 1	Operand added to postfix expression.
9	Read)	/	2 3 * 2 1 -	) is encountered; operators are popped until (is found. (is discarded.
10	Read +	+	2 3 * 2 1 - /	Operator + has lower precedence than /, so / is popped to postfix, and + is pushed onto the stack.
11	Read 5	+	2 3 * 2 1 - / 5	Operand added to postfix expression.
12	Read *	* +	2 3 * 2 1 - / 5	Operator * pushed onto stack.
13	Read 3	* +	2 3 * 2 1 - / 5 3 *	Operand added to postfix expression.
14	End of Expression		2 3 * 2 1 - / 5 3 * +	Pop remaining operators to pos

PREPARD BY:RUDRAM T KSHATRI

Evaluation of postfix notations:-

Step	Action	Stack	Explanation
1	Read `2`	`2`	Operand `2` pushed onto stack.
2	Read `3`	`2 3`	Operand `3` pushed onto stack.
3	Encounter `*`	`6`	Pop `3` and `2`, compute `2 * 3 = 6`, push result.
4	Read `2`	`6 2`	Operand `2` pushed onto stack.
5	Read `1`	`6 2 1`	Operand `1` pushed onto stack.
6	Encounter `-`	`6 1`	Pop `1` and `2`, compute `2 - 1 = 1`, push result.
7	Encounter `/`	`6`	Pop `1` and `6`, compute `6 / 1 = 6`, push result.
8	Read `5`	`6 5`	Operand `5` pushed onto stack.
9	Read `3`	`6 5 3`	Operand `3` pushed onto stack.
10	Encounter `*`	`6 15`	Pop `3` and `5`, compute `5 * 3 = 15`, push result.
11	Encounter `+`	`21`	Pop `15` and `6`, compute `6 + 15 = 21`, push result.

28. Transform the following expression to postfix and evaluate postfix expression by assuming (4-5 Marks)

A=1,B=2,C=3,D=4,E=6,F=6,G=1,I=3 and J=3.

INFIX- A + B – C * D / E + F * G / (I + J)

⇨ **<u>Step 1: Convert Infix to Postfix Notation</u>**

Conversion Process

1. **Infix Expression**: A + B - C * D / E + F * G / (I + J)
2. **Apply Operator Precedence and Associativity Rules**:
   - **Operators**: +, -, *, /
   - **Precedence**: * and / have higher precedence than + and -
   - **Associativity**: Left-to-right for all operators.

<u>Conversion Table</u>

Step	Action	Stack	Postfix Expression	Explanation
1	Read A		A	Operand A added to postfix expression.
2	Read +	+	A	Operator + pushed onto the stack.
3	Read B	+	A B	Operand B added to postfix expression.

PREPARD BY:RUDRAM T KSHATRI

Step	Action	Stack	Postfix Expression	Explanation
4	Read -	-	A B +	Pop + (higher precedence), add to postfix. Push - to stack.
5	Read C	-	A B + C	Operand C added to postfix expression.
6	Read *	- *	A B + C	Operator * pushed onto stack (higher precedence than -).
7	Read D	- *	A B + C D	Operand D added to postfix expression.
8	Read /	- /	A B + C D *	Pop * (higher precedence), add to postfix. Push / to stack.
9	Read E	- /	A B + C D * E	Operand E added to postfix expression.
10	Read +	+	A B + C D * E / -	Pop / (same precedence), add to postfix. Pop - (lower precedence), add to postfix. Push + to stack.
11	Read F	+	A B + C D * E / - F	Operand F added to postfix expression.
12	Read *	+ *	A B + C D * E / - F	Operator * pushed onto stack.
13	Read G	+ *	A B + C D * E / - F G	Operand G added to postfix expression.
14	Read /	+ /	A B + C D * E / - F G *	Pop * (same precedence), add to postfix. Push / to stack.
15	Read (	+ / (	A B + C D * E / - F G * /	(pushed onto stack.
16	Read I	+ / (	A B + C D * E / - F G * / I	Operand I added to postfix expression.
17	Read +	+ / (+	A B + C D * E / - F G * / I	+ pushed onto stack (inside ().
18	Read J	+ / (+	A B + C D * E / - F G * / I J	Operand J added to postfix expression.
19	Read)	+	A B + C D * E / - F G * / I J +	Pop + (until (is found), add to postfix. Pop ((discard).
20	End of Expression		A B + C D * E / - F G * / I J +	Pop remaining operators from stack and add to postfix.

Postfix Expression: A B + C D * E / - F G * / I J +

PREPARD BY:RUDRAM T KSHATRI

Step 2: Evaluate Postfix Expression

Evaluation Table

Step	Action	Stack	Explanation
1	Read A = 1	1	Operand A (1) pushed onto stack.
2	Read B = 2	1 2	Operand B (2) pushed onto stack.
3	Encounter +	3	Pop 2 and 1, compute 1 + 2 = 3, push result.
4	Read C = 3	3 3	Operand C (3) pushed onto stack.
5	Read D = 4	3 3 4	Operand D (4) pushed onto stack.
6	Encounter *	3 12	Pop 4 and 3, compute 3 * 4 = 12, push result.
7	Read E = 6	3 12 6	Operand E (6) pushed onto stack.
8	Encounter /	3 2	Pop 6 and 12, compute 12 / 6 = 2, push result.
9	Encounter -	1	Pop 2 and 3, compute 3 - 2 = 1, push result.
10	Read F = 6	1 6	Operand F (6) pushed onto stack.
11	Read G = 1	1 6 1	Operand G (1) pushed onto stack.
12	Encounter *	1 6	Pop 1 and 6, compute 6 * 1 = 6, push result.
13	Encounter /	1 1	Pop 6 and 1, compute 6 / 1 = 6, push result.
14	Read I = 3	1 6 3	Operand I (3) pushed onto stack.
15	Read J = 3	1 6 3 3	Operand J (3) pushed onto stack.
16	Encounter +	1 6 6	Pop 3 and 3, compute 3 + 3 = 6, push result.
17	Encounter /	1 1	Pop 6 and 6, compute 6 / 6 = 1, push result.
18	Encounter +	2	Pop 1 and 1, compute 1 + 1 = 2, push result.

- **Postfix Expression**: A B + C D * E / - F G * / I J +
- **Evaluation Result**: 2

PREPARD BY:RUDRAM T KSHATRI

29. Differentiate between LIFO and FIFO access mechanism. (3 Marks)

Aspect	LIFO (Last In, First Out)	FIFO (First In, First Out)
Definition	The most recently added element is the first to be removed.	The oldest added element is the first to be removed.
Data Structure	Implemented using a stack.	Implemented using a queue.
Order of Access	Last element added is accessed first.	First element added is accessed first.
Example	Function call stack, undo mechanism in text editors, web browser history.	Print spooling, task scheduling in operating systems, queue of customer service requests.
Insertion	New elements are pushed onto the top of the stack.	New elements are enqueued at the rear of the queue.
Removal	Elements are popped from the top of the stack.	Elements are dequeued from the front of the queue.
Access Time	Typically O(1) for both push and pop operations.	Typically O(1) for both enqueue and dequeue operations.
Use Case	Useful in scenarios requiring backtracking or managing function calls and returns.	Useful in scenarios requiring maintaining order, such as processing tasks in the order they arrive.

30. How linked list is better compared to stack, queue and array? Explain with concept of dynamic memory allocation. (3-4 Marks)

❖ **<u>Linked List</u>**

⇨ A linked list is a linear data structure where each element (node) points to the next element. It is particularly useful when dynamic memory allocation is needed.

⇨ **<u>Advantages Over Array</u>**

1. **Dynamic Size**:
   - **Linked List**: Can grow or shrink in size dynamically as elements are added or removed. This flexibility is due to its use of dynamic memory allocation.
   - **Array**: Has a fixed size once defined. Changing the size requires creating a new array and copying the old elements over.
2. **Efficient Insertion/Deletion**:
   - **Linked List**: Insertion and deletion of elements are efficient (O(1) time complexity) if the position is known. This is because only the pointers need to be adjusted.
   - **Array**: Insertion and deletion can be costly (O(n) time complexity) as elements may need to be shifted to maintain order.

PREPARD BY:RUDRAM T KSHATRI

3. **Memory Utilization**:
    - **Linked List**: Allocates memory as needed for each node. This can be more efficient in terms of memory usage if the number of elements varies greatly.
    - **Array**: May waste memory if the array is not fully utilized, or it may need to be resized if it runs out of space.

⇨ **Comparison with Stack and Queue**

1. **Stack**:
    - **Linked List**: Implements a stack efficiently where elements are pushed and popped from one end (top). The linked list allows for dynamic resizing and avoids the limitation of a fixed size.
    - **Stack (Array-based)**: Requires a fixed size or resizing, which can be inefficient if the stack grows beyond its initial capacity.
2. **Queue**:
    - **Linked List**: Implements a queue efficiently with dynamic size. Elements can be enqueued at the rear and dequeued from the front without resizing.
    - **Queue (Array-based)**: Similar to stacks, array-based queues may need resizing and can suffer from inefficiencies if the queue grows beyond its initial capacity.

⇨ **Dynamic Memory Allocation**

- **Linked List**:
    - Uses dynamic memory allocation where each node is allocated memory as needed. This means that memory is allocated for each element dynamically, allowing the list to grow or shrink without a predefined size.
    - **Advantage**: More flexible memory usage, as it only uses the amount of memory required for the current number of elements.
- **Array**:
    - Uses static memory allocation for a fixed size. To accommodate more elements, a new array must be created, and the old elements copied over (in the case of resizing).
    - **Disadvantage**: Can lead to wasted memory or the need for expensive resizing operations.

31. In which type of scenario, linear queue (simple queue) is better than circular queue? (2-3 Marks)

⇨ A linear queue (simple queue) is often better than a circular queue in the following scenarios:

1. **Fixed Size and It's Usage**:
    - **Scenario**: When the number of elements is fixed and predictable, and there's no concern about wrapping around or the queue getting full prematurely.
    - **Reason**: In such cases, the overhead of managing the circular structure is unnecessary, and a simple queue can be more straightforward to implement and manage.
2. **Simplicity of Implementation**:
    - **Scenario**: When simplicity and ease of implementation are prioritized, such as in educational contexts or simple applications where the overhead of circular queue logic isn't justified.
    - **Reason**: Linear queues are simpler to implement as they do not require managing the wrap-around of the front and rear pointers. This makes them easier to understand and use in scenarios where performance and efficient space utilization are less critical.

PREPARD BY RUDRAM TKSHATRI

3. **Non-High Performance Requirements**:
   - **Scenario**: In applications where high performance and efficient space utilization are not crucial, and where the queue will not become full or require frequent wrap-arounds.
   - **Reason**: For less performance-critical applications, the simpler linear queue might be sufficient and easier to maintain without the complexity of circular queue logic.

32. After evaluation of 3,5,4,*,+ result is ? (2-3Marks)

Step	Action	Operands	Operator	Result	Stack
1	Read `3`	-	-	-	`[3]`
2	Read `5`	-	-	-	`[3, 5]`
3	Read `4`	-	-	-	`[3, 5, 4]`
4	Read `*`	`5`, `4`	`*`	`20`	`[3, 20]`
5	Read `+`	`3`, `20`	`+`	`23`	`[23]`

Final result: 23

33. What will be the value of Front and Rear pointers when Queue is empty? (2 Marks)

⇨ When a queue is empty, the values of the **Front** and **Rear** pointers typically depend on the implementation, but common conventions are:

1. **Front Pointer**:
   - **Value**: Often set to `-1` or `null` to indicate that there are no elements in the queue.
2. **Rear Pointer**:
   - **Value**: Similarly, often set to `-1` or `null` for the same reason.

Hence:

- **Front**: `-1` or `null`
- **Rear**: `-1` or `null`

⇨ These values indicate that the queue is empty and no elements are present.

PREPARD BY.RUDRAM T KSHATRI

34. Apply selection sort algorithm on following input. 12, 29, 25, 8, 32, 17 , 40,Explain step by step. (4-5 Marks)

Pass	Unsorted Portion	Minimum Element	Swap with	Array after Pass
1	`[12, 29, 25, 8, 32, 17, 40]`	`8` (Index 3)	`12` (Index 0)	`[8, 29, 25, 12, 32, 17, 40]`
2	`[29, 25, 12, 32, 17, 40]`	`12` (Index 2)	`29` (Index 1)	`[8, 12, 25, 29, 32, 17, 40]`
3	`[25, 29, 32, 17, 40]`	`17` (Index 5)	`25` (Index 2)	`[8, 12, 17, 29, 32, 25, 40]`
4	`[29, 32, 25, 40]`	`25` (Index 2)	`29` (Index 3)	`[8, 12, 17, 25, 32, 29, 40]`
5	`[32, 29, 40]`	`29` (Index 1)	`32` (Index 4)	`[8, 12, 17, 25, 29, 32, 40]`
6	`[32, 40]`	Already sorted	-	`[8, 12, 17, 25, 29, 32, 40]`

⇨ **<u>Explanation:</u>**

1. **Pass 1**:
   - Find the minimum element in the entire array (8 at index 3).
   - Swap 8 with 12 (the first element).
2. **Pass 2**:
   - Find the minimum element in the remaining unsorted portion (12 at index 2).
   - Swap 12 with 29 (the first element of the unsorted portion).
3. **Pass 3**:
   - Find the minimum element in the remaining unsorted portion (17 at index 5).
   - Swap 17 with 25 (the first element of the unsorted portion).
4. **Pass 4**:
   - Find the minimum element in the remaining unsorted portion (25 at index 2).
   - Swap 25 with 29 (the first element of the unsorted portion).
5. **Pass 5**:
   - Find the minimum element in the remaining unsorted portion (29 at index 1).
   - Swap 29 with 32 (the first element of the unsorted portion).
6. **Pass 6**:
   - The remaining portion is already sorted, so no changes are needed.

⇨ **The final sorted array is [8, 12, 17, 25, 29, 32, 40].**

PREPARD BY:RUDRAM TKSHATRI

35. Write an algorithm for bubble sort. Apply it on random 8 input data. (4-5 Marks)

⇨ **Algorithm**:

1. **Start**: Begin with the first element of the array.
2. **Compare Adjacent Elements**: Compare each pair of adjacent elements.
3. **Swap if Needed**: If the first element of the pair is greater than the second, swap them.
4. **Repeat**: Continue comparing and swapping adjacent elements until the end of the array.
5. **Pass Completion**: After each pass through the array, the largest element will be in its correct position.
6. **Reduce Array Size**: Reduce the size of the unsorted portion of the array by one, and repeat the process until the array is fully sorted.
7. **End**: The algorithm finishes when no more swaps are needed during a pass.

⇨ **Apply Bubble Sort on Random Data**

Input Data: [34, 7, 23, 32, 5, 62, 32, 10]

Pass	Comparison	Swaps	Array after Pass
1	Compare 34 and 7 → Swap	[7, 34, 23, 32, 5, 62, 32, 10]	7 and 34 swapped
	Compare 34 and 23 → Swap	[7, 23, 34, 32, 5, 62, 32, 10]	23 and 34 swapped
	Compare 34 and 32 → Swap	[7, 23, 32, 34, 5, 62, 32, 10]	32 and 34 swapped
	Compare 34 and 5 → Swap	[7, 23, 32, 5, 34, 62, 32, 10]	5 and 34 swapped
	Compare 34 and 62 → No Swap	[7, 23, 32, 5, 34, 62, 32, 10]	No change
	Compare 62 and 32 → Swap	[7, 23, 32, 5, 34, 32, 62, 10]	32 and 62 swapped
	Compare 62 and 10 → Swap	[7, 23, 32, 5, 34, 32, 10, 62]	10 and 62 swapped
Pass	**Comparison**	**Swaps**	**Array after Pass**
2	Compare 7 and 23 → No Swap	[7, 23, 32, 5, 34, 32, 10, 62]	No change
	Compare 23 and 32 → No Swap	[7, 23, 32, 5, 34, 32, 10, 62]	No change
	Compare 32 and 5 → Swap	[7, 23, 5, 32, 34, 32, 10, 62]	5 and 32 swapped
	Compare 32 and 34 → No Swap	[7, 23, 5, 32, 34, 32, 10, 62]	No change
	Compare 34 and 32 → Swap	[7, 23, 5, 32, 32, 34, 10, 62]	32 and 34 swapped
	Compare 34 and 10 → Swap	[7, 23, 5, 32, 32, 10, 34, 62]	10 and 34 swapped

PREPARD BY:RUDRAM T KSHATRI

Pass	Comparison	Swaps	Array after Pass
3	Compare 7 and 23 → No Swap	[7, 23, 5, 32, 32, 10, 34, 62]	No change
	Compare 23 and 5 → Swap	[7, 5, 23, 32, 32, 10, 34, 62]	5 and 23 swapped
	Compare 23 and 32 → No Swap	[7, 5, 23, 32, 32, 10, 34, 62]	No change
	Compare 32 and 32 → No Swap	[7, 5, 23, 32, 32, 10, 34, 62]	No change
	Compare 32 and 10 → Swap	[7, 5, 23, 32, 10, 32, 34, 62]	10 and 32 swapped

Pass	Comparison	Swaps	Array after Pass
4	Compare 7 and 5 → Swap	[5, 7, 23, 32, 10, 32, 34, 62]	5 and 7 swapped
	Compare 7 and 23 → No Swap	[5, 7, 23, 32, 10, 32, 34, 62]	No change
	Compare 23 and 32 → No Swap	[5, 7, 23, 32, 10, 32, 34, 62]	No change
	Compare 32 and 10 → Swap	[5, 7, 23, 10, 32, 32, 34, 62]	10 and 32 swapped

Pass	Comparison	Swaps	Array after Pass
5	Compare 5 and 7 → No Swap	[5, 7, 23, 10, 32, 32, 34, 62]	No change
	Compare 7 and 23 → No Swap	[5, 7, 23, 10, 32, 32, 34, 62]	No change
	Compare 23 and 10 → Swap	[5, 7, 10, 23, 32, 32, 34, 62]	10 and 23 swapped

Pass	Comparison	Swaps	Array after Pass
6	Compare 5 and 7 → No Swap	[5, 7, 10, 23, 32, 32, 34, 62]	No change
	Compare 7 and 10 → No Swap	[5, 7, 10, 23, 32, 32, 34, 62]	No change

Pass	Comparison	Swaps	Array after Pass
7	Compare 5 and 7 → No Swap	[5, 7, 10, 23, 32, 32, 34, 62]	No change

PREPARD BY:RUDRAM T KSHATRI

36. Write Merge Sort algorithm. Apply the algorithm to the following elements: 10,5,28, 7, 39,310, 55,15,1.(4-5 Marks)

⇨ **Algorithm**:

1. **Divide**: Split the array into two halves.
2. **Conquer**: Recursively sort each half.
3. **Combine**: Merge the two sorted halves to produce a sorted array.

⇨ **Apply Merge Sort to [10, 5, 28, 7, 39, 310, 55, 15, 1]**

1. **Initial Array**: [10, 5, 28, 7, 39, 310, 55, 15, 1]
2. **Divide**:
   - Left half: [10, 5, 28, 7, 39]
   - Right half: [310, 55, 15, 1]
3. **Recursively sort each half**:

Sorting [10, 5, 28, 7, 39]:

- Divide into [10, 5] and [28, 7, 39]

Sorting [10, 5]:

- Divide into [10] and [5]
- Merge [10] and [5] → [5, 10]

Sorting [28, 7, 39]:

- Divide into [28] and [7, 39]

Sorting [7, 39]:

- Divide into [7] and [39]
- Merge [7] and [39] → [7, 39]
- Merge [28] and [7, 39] → [7, 28, 39]
- Merge [5, 10] and [7, 28, 39] → [5, 7, 10, 28, 39]

Sorting [310, 55, 15, 1]:

- Divide into [310, 55] and [15, 1]

Sorting [310, 55]:

- Divide into [310] and [55]
- Merge [310] and [55] → [55, 310]

PREPARD BY:RUDRAM T KSHATRI

Sorting [15, 1]:

- Divide into [15] and [1]
- Merge [15] and [1] → [1, 15]
- Merge [55, 310] and [1, 15] → [1, 15, 55, 310]

4. **Merge the sorted halves**:
   - Merge [5, 7, 10, 28, 39] and [1, 15, 55, 310]
   - Compare and merge:
   - [1], [5], [7], [10], [15], [28], [39], [55], [310]

Final Sorted Array: [1, 5, 7, 10, 15, 28, 39, 55, 310]

PREPARD BY:RUDRAM T KSHATRI